

Spring Boot API & Services

RESTful Endpoints, DTO Architecture & Domain Business Layer

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume establishes the backend application engineering standards of PropLedge. Built with Spring Boot 3.3.4 and Java 21, it details dual-routed RESTful controllers, strict Data Transfer Object (DTO) isolation, zero-reflection builder mappers, Jakarta Bean Validation, centralized RFC 7807 exception handling, and decoupled domain business services.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Dual RESTful API Architecture	Backwards-compatible /api and /api/v1 routing standards
03	The Life of an HTTP Request	End-to-end execution path from socket to PostgreSQL and back
04	Why DTOs are Mandatory	Defense against mass-assignment, circular recursion, and lazy errors
05	Mapper Design & Zero-Reflection	Lombok builder pattern vs MapStruct compile-time mappers
06	Jakarta Bean Validation Engine	@Valid, @NotNull, @DecimalMin, and custom constraint enforcement
07	Controller Layer Architecture	Standard envelopes (ApiResponse<T>, PagedResponse<T>) and HTTP codes
08	Centralized Exception Handling	GlobalExceptionHandler conforming to RFC 7807 problem details
09	Domain Service Layer & Coordinators	Decoupling business transactions from transport protocols
10	Spring Boot & REST Interview Q&A	Senior engineering interview scenarios and design tradeoffs

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedge production repository.

2. DUAL RESTFUL API ARCHITECTURE & ROUTING

BACKWARDS COMPATIBILITY VIA /API AND /API/V1 DUAL-ROUTING

1. The Enterprise Versioning Dilemma

In enterprise SaaS platforms serving multiple client generations (web apps, native mobile apps, and third-party partner ERPs), breaking API contracts causes catastrophic business disruption. PropLedger implements **Dual-Routing** across all 13 controllers:

```
// CODE LISTING: JAVA 21 – DUAL-ROUTING REST CONTROLLER

// Dual-Routing Controller Declaration in PropertyController.java
@RestController
@RequestMapping({" /api/properties", " /api/v1/properties"})
@RequiredArgsConstructor
public class PropertyController {
    // Both endpoints /api/properties and /api/v1/properties map to this identical handler!
}
```

2. Controller Inventory & Base Paths

Domain Subsystem	Primary Controller	Dual Base Path	Primary Operations
Authentication	AuthController	/api/v1/auth	login, register, me
Asset Management	PropertyController	/api/v1/properties	CRUD properties, asset summary
Space Inventory	UnitController	/api/v1/units	Unit specs, availability filter
Resident Directory	TenantController	/api/v1/tenants	Onboard residents, credit screening
Lease Lifecycle	LeaseController	/api/v1/leases	Execute, renew, early-terminate
Accounts Receivable	InvoiceController	/api/v1/invoices	Monthly billing, line items
Payment Settlement	PaymentController	/api/v1/payments	Multi-invoice split allocations

3. THE LIFE OF AN HTTP REQUEST: END-TO-END

FROM TCP SOCKET TO DATABASE ENGINE AND BACK

1. Detailed Execution Pipeline

- 1. TCP Handshake & Worker Allocation: Tomcat accepts the connection and assigns a thread from its worker pool.
- 2. Security Filter Traversal: JwtAuthenticationFilter extracts Bearer token, validates HS256 signature, and populates SecurityContextHolder.
- 3. DispatcherServlet Dispatch: DispatcherServlet queries RequestMappingHandlerMapping to match URI and HTTP method to controller.
- 4. Jackson Deserialization: MappingJackson2HttpMessageConverter reads HTTP body and maps JSON to Request DTO.
- 5. Jakarta Bean Validation: Validator checks @Valid constraints (@NotBlank, @DecimalMin). If invalid, throws exception.
- 6. CGLIB Transaction Interception: TransactionInterceptor begins transaction, borrows connection from HikariCP, and binds thread.
- 7. Service Execution & Mapping: Service applies domain rules, invokes JPA repositories, and transforms entities into Response DTOs.
- 8. Transaction Commit: TransactionInterceptor commits transaction and releases HikariCP connection.
- 9. Serialization & Response: DispatcherServlet serializes ApiResponse<T> envelope to JSON and writes HTTP 200/201.

4. WHY DATA TRANSFER OBJECTS (DTOS) ARE MANDATORY

PREVENTING MASS-ASSIGNMENT, RECURSION & LAZY ERRORS

1. The Dangers of Exposing JPA Entities Directly

Exposing JPA Entities directly through REST controllers is a fatal security and architectural anti-pattern:

- 1. Mass-Assignment / Over-Posting: If an entity is accepted directly in `@RequestBody`, an attacker can supply uneditable fields (e.g. `balanceDue: 0.00`, `status: 'PAID'`) and compromise the ledger.
- 2. Infinite Recursion (`StackOverflowError`): Bidirectional associations (`Property -> Units -> Property`) crash Jackson serialization during JSON generation.
- 3. `LazyInitializationException`: Accessing uninitialized lazy associations outside transaction boundaries crashes the serializer with 'could not initialize proxy - no Session'.
- 4. API Contract Tight Coupling: Renaming an internal database column immediately breaks external client integrations.

2. DTO Architecture in PropLedger

PropLedger strictly separates Request DTOs (only permitting client-writable fields) from Response DTOs (safe, flattened client representations):

DTO Category	Sample Class	Key Responsibilities	Validation Mechanism
Request DTO	PropertyCreateDto	Carries user input; zero persistence logic	Jakarta <code>@Valid</code> (<code>@NotBlank</code> , <code>@Size</code>)
Response DTO	PropertyResponseDto	Client view; flattened; circular-free	Lombok <code>@Builder</code> , immutable fields
Summary DTO	RentRollDto	Projection for high-performance reporting	Constructed via JPQL <code>SELECT new ...</code>

5. MAPPER DESIGN & ZERO-REFLECTION BUILDERS

TYPE-SAFE TRANSFORMATIONS VS RUNTIME REFLECTION OVERHEAD

1. Why ModelMapper Fails in High-Throughput Systems

Reflection-based mappers (like ModelMapper or Dozer) use runtime reflection to inspect class properties on every request. In high-throughput systems, reflection incurs severe CPU overhead and fails silently when field names mismatch.

2. PropLedger's Manual Builder Mapper Pattern

PropLedger implements explicit, type-safe builder mappers offering zero-reflection runtime speed and step-through debugging:

```
// CODE LISTING: JAVA 21 – ZERO-REFLECTION BUILDER MAPPING
```

```
// Manual Type-Safe Mapping in PropertyServiceImpl.java
private PropertyResponseDto mapToResponseDto(Property property) {
    return PropertyResponseDto.builder()
        .id(property.getId())
        .propertyCode(property.getPropertyCode())
        .name(property.getName())
        .propertyType(property.getPropertyType().name())
        .ownerId(property.getOwner() != null ? property.getOwner().getId() : null)
        .ownerName(property.getOwner() != null ? property.getOwner().getCompanyName() : null)
        .addressLine1(property.getAddressLine1())
        .city(property.getCity())
        .state(property.getState())
        .postalCode(property.getPostalCode())
        .yearBuilt(property.getYearBuilt())
        .totalAreaSqft(property.getTotalAreaSqft())
        .totalUnits(property.getUnits() != null ? property.getUnits().size() : 0)
        .createdAt(property.getCreatedAt())
        .build();
}
```

6. JAKARTA BEAN VALIDATION ENGINE

FAIL-FAST INPUT SANITATION & CUSTOM CONSTRAINT RULES

1. Declarative Field Validation

All incoming Request DTOs are validated at the controller boundary before the service layer is reached. If invalid, Spring raises `MethodArgumentNotValidException`:

```
// CODE LISTING: JAVA 21 – JAKARTA VALIDATION ANNOTATIONS

// PropertyCreateDto.java: Declarative Input Validation
@Getter @Setter @Builder @NoArgsConstructor @AllArgsConstructor
public class PropertyCreateDto {
    @NotBlank(message = "Property code is mandatory")
    @Pattern(regexp = "[A-Z0-9-]+$", message = "Property code must be alphanumeric with hyphens")
    private String propertyCode;

    @NotBlank(message = "Property name cannot be blank")
    @Size(min = 2, max = 150, message = "Name must be between 2 and 150 characters")
    private String name;

    @NotNull(message = "Owner ID cannot be null")
    private Long ownerId;

    @NotNull(message = "Year built is mandatory")
    @Min(value = 1800, message = "Year built must be after 1800")
    private Integer yearBuilt;
}
```

2. Standard Annotation Reference

Annotation	Validation Rule	Primary PropLedger Use Case
@NotBlank	Not null, not empty, and not whitespace only	Username, titles, property codes
@DecimalMin	Asserts minimum numeric currency value	Rent amounts (@DecimalMin('0.01'))
@FutureOrPresent	Asserts date is today or in the future	Lease start dates
@Email	Validates RFC 5322 email address format	Resident and vendor email addresses

7. CONTROLLER ARCHITECTURE & RESPONSE ENVELOPES

STANDARDIZED REST PAYLOADS & HTTP STATUS SEMANTICS

1. Standard Response Envelope (ApiResponse<T>)

PropLedger standardizes all single-entity REST responses into a uniform envelope:

```
// CODE LISTING: JAVA 21 – APIRESPONSE ENVELOPE STRUCTURE

public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;
    private Instant timestamp;

    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(true, message, data, Instant.now());
    }
}

// JSON Output Example:
// { "success": true, "message": "Operation successful", "data": { ... }, "timestamp": "..." }
```

2. Paginated Response Envelope (PagedResponse<T>)

Field Name	Type	Description & Client Usage
content	List<T>	Array of records for the requested page slice.
pageNumber	int	Current 0-indexed page number.
pageSize	int	Number of records requested per page.
totalElements	long	Total count of records matching filter across entire table.
totalPages	int	Calculated total pages (ceil(totalElements / pageSize)).
last	boolean	True if current page is the final page.

8. CENTRALIZED EXCEPTION HANDLING (RFC 7807)

GLOBAL CONTROLLER ADVICE & FIELD ERROR DESERIALIZATION

1. RFC 7807 Problem Details Standard

Unhandled server errors must never leak stack traces to clients. PropLedger implements a centralized `@ControllerAdvice` returning standardized problem payloads:

// CODE LISTING: JAVA 21 – GLOBALEXCEPTIONHANDLER IMPLEMENTATION

```
// GlobalExceptionHandler.java: RFC 7807 Problem Details
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationErrors(MethodArgumentNotValidException ex) {
        List<FieldErrorItem> fieldErrors = ex.getBindingResult().getFieldErrors().stream()
            .map(err -> new FieldErrorItem(err.getField(), err.getDefaultMessage(), err.getRejectedValue()))
            .toList();

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
            ErrorResponse.builder()
                .status(400).error("Bad Request")
                .message("Validation failed for one or more fields")
                .fieldErrors(fieldErrors).timestamp(Instant.now()).build()
        );
    }
}
```


9. DOMAIN SERVICE LAYER & BUSINESS COORDINATORS

DECOUPLING BUSINESS LOGIC FROM WEB TRANSPORTS

1. The Role of Domain Services

The Service Layer in PropLedge represents the authoritative business transaction coordinator:

- 1. Transaction Boundaries: Coordinates multi-repository operations under atomic `@Transactional` scopes.
- 2. Business Rule Enforcement: Validates operational invariants (e.g. verifying unit is vacant before lease creation).
- 3. Transport Independence: Services have zero dependencies on `HttpServletRequest` or HTTP concepts, allowing identical reuse in CLI jobs or message queues.
- 4. Audit Dispatch: Dispatches immutable audit records upon successful entity mutation.

10. SPRING BOOT & REST ARCHITECTURE INTERVIEW Q&A

SENIOR BACKEND ENGINEERING INTERVIEW SCENARIOS

1. High-Yield Backend Interview Questions

Question 1: What is the difference between @RestController and @Controller?

Answer: @Controller is designed for traditional Spring MVC web applications returning HTML view templates (JSP, Thymeleaf). @RestController is a convenience meta-annotation combining @Controller and @ResponseBody, instructing Spring to serialize method return values directly into HTTP response bodies (JSON) using Jackson.

Question 2: How do you prevent N+1 query cascades when using Spring Data JPA?

Answer: 1) Declare associations as FetchType.LAZY. 2) Use 'JOIN FETCH' in JPQL queries to load parent and child associations in a single SQL query. 3) Configure 'hibernate.default_batch_fetch_size = 30' in application.properties, allowing Hibernate to batch-fetch lazy collections using an 'IN (?, ?, ...)' clause rather than individual queries.

Question 3: Why should services return DTOs rather than JPA Entities?

Answer: Returning entities leaks persistence concerns into the presentation layer, risks LazyInitializationExceptions if associations are accessed outside transaction boundaries, and exposes sensitive or circular data. Returning immutable DTOs guarantees clean separation of concerns and deterministic API contracts.

BACKEND STAFF ENGINEER INTERVIEW TIP

Articulating how you eliminated LazyInitializationException and N+1 query cascades demonstrates deep mastery of Spring Data JPA and Hibernate.